ITE | Assignment | F25
Intelligent Algorithms | BIA601

# Makespan Optimization for Task Scheduling on Heterogeneous Machines

Supervised by: Dr. Issam Salman

Prepared by the Students:

| Student name | University ID | Class | Primary Responsibilities |
|---|---|---|---|
| ماري حسين | mary_174155 | C3 | تصميم خطة التجارب للسيناريوهات الثلاثة (Small, Medium, Large) |
| محمد سليمان | mohmmad_220248 | C3 | صياغة نموذج الـ MIP الرياضي، دوال الهدف، وتعريف القيود |
| جنى الشرابي | jana_246118 | C2 | برمجة وتطوير خوارزمية مستعمرة النمل (ACO) والقاعدة الأساسية (Greedy) |
| ملدا قدور | mulda_197917 | C1 | تطوير، برمجة، وضبط معاملات خوارزمية سرب الجسيمات (PSO) |
| حلا الشرابي | hala_173382 | C5 | تطوير، برمجة، وضبط معاملات الخوارزمية الجينية (GA) |
| هبة وتر | hiba_190858 | C2 | تنفيذ الـ 54 تجربة، تجميع السجلات الخام، وحساب الانحراف المعياري |
| تسنيم دركل | tasnim_119270 | C6 | تحليل سلوك التقارب (Convergence) وتصميم الرسوم البيانية |
| رنيم الكسم | raneem_151385 | C5 | صياغة الملخص التنفيذي، الخاتمة، Synthesis وتنسيق التقرير الأكاديمي نهائياً |

## Table of Contents

1. Executive Summary
2. Problem Formulation & Mathematical Model
    2.1 Notation and Definitions
    2.2 Mathematical Formulation
3. System Architecture & Algorithmic Implementation
    3.1 Overview of Project Structure
    3.2 Evaluation & Fitness Function
    3.3 Metaheuristics Implementation (GA, PSO, and ACO Core Code Snippets)
        A. Genetic Algorithm (GA) Core Implementation
        B. Particle Swarm Optimization (PSO) Core Implementation
        C. Ant Colony Optimization (ACO) Core Implementation
    3.4 Core Code Snippets
4. Experimental Framework & Results Summary
    4.1 Aggregate Performance Across 54 Benchmark Runs
5. Discussion & Visualization Analysis
    5.1 Solution Quality and Makespan Optimization
    5.2 Convergence Trajectories Across Problem Scales
    5.3 Computational Complexity and Scalability
6. Terminal (Proof of Execution & Server Initialization)
7. Conclusion & Future Work Recommendations
    7.1 Recommendations for Future Research

# Project Information & Metadata

| Item | Verified project detail |
|---|---|
| Project title | Makespan Optimization for Independent Task Scheduling on Heterogeneous Machines |
| Problem class | Unrelated parallel-machine scheduling, R \|\| Cmax |
| Objective | Minimize the latest machine completion time (makespan), in minutes |
| Algorithms | GA, PSO, ACO, plus deterministic Greedy baseline |
| Implementation | Python 3.10+; custom algorithms in scheduler.py |
| Dataset | Seeded synthetic workshop processing times; not company measurements |
| Reproducibility | run_experiments.py, tests/, results/, and README.md |
| Experiment scope | 3 benchmark sizes; 6 independent runs per algorithm and size; 54 stochastic runs |
| Live Web Dashboard | https://makespan-optimization |
| GitHub | Repository:https://github.com/Maryhousen/Makespan-Optimization-BIA601/tree/main/Optimization_Project_F25 |

# 1. Executive Summary

This project evaluates independent, non-preemptive task scheduling on heterogeneous machines. Each task must be assigned to exactly one machine, and its processing time depends on the selected machine. The scheduling objective is to minimize makespan: the maximum total processing load across machines. Three custom metaheuristics—Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO)—are compared with an LPT-like Greedy baseline. The study uses three reproducible synthetic workshop instances: 20 tasks on 3 machines, 50 on 5, and 100 on 8. Each metaheuristic was executed six times per instance with 80 iterations and a search population, swarm or ant count of 32, yielding 54 stochastic benchmark runs. The saved experiment files show that GA obtained the lowest mean makespan for Small (118.65 min) and Large (220.75 min), whereas ACO was strongest for Medium (200.57 min). The largest mean improvement over Greedy was 7.67% (GA, Small). PSO did not improve over Greedy for the Large instance under the selected configuration. These are observations for the three tested instances and seeds; none constitutes proof of global optimality.

| Scale | Greedy (min) | Best mean (min) | Algorithm | Gain vs Greedy |
|---|---|---|---|---|
| Small: 20 × 3 | 128.51 | 118.65 | GA | 7.67% |
| Medium: 50 × 5 | 206.84 | 200.57 | ACO | 3.03% |
| Large: 100 × 8 | 222.69 | 220.75 | GA | 0.87% |

# 2. Problem Formulation & Mathematical Model

The problem belongs to the unrelated parallel-machine scheduling family (R || Cmax). Jobs are independent; machines may process in parallel but each machine executes its assigned jobs sequentially. Task interruption, release dates, precedences and sequence-dependent changeover times are not modeled

## 2.1 Notation and Definitions

- n: number of tasks; m: number of machines.
- p[i, j] (minutes): processing time for task i on machine j; the ETC matrix has shape n × m.
- a[i] ∈ {0, ..., m-1}: integer machine assignment for task i in the Python implementation
- x[i, j] ∈ {0, 1}: binary formulation variable indicating that task i uses machine j.
- L[j]: summed processing time for machine j; Cmax = max_j L[j]

## 2.2 Mathematical Formulation

A compact mixed-integer model is:

$$Minimize C_{\max}$$

**Subject to:**

$$\sum_{j=1}^{m} x_{ij} = 1 \quad \forall i \in \{1, \ldots, n\}$$

$$\sum_{i=1}^{n} p_{ij} x_{ij} \leq C_{\max} \quad \forall j \in \{1, \ldots, m\}$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in \{1, \ldots, n\}, \forall j \in \{1, \ldots, m\}$$

$$C_{\max} \geq 0$$

The first constraint assigns every task exactly once. The second bounds the load of each machine by Cmax. An appropriate computable lower bound is LB = max[(sum_i min_j p[i,j]) / m, max_i min_j p[i,j]]. It provides a reference but is not a verified optimum for these benchmark instances

# 3. System Architecture & Algorithmic Implementation

This section outlines the software architecture of the scheduling framework and details the core algorithmic implementations used to solve the unrelated parallel machine scheduling problem ( $R \parallel C_{\max}$ ). The source code is organized into modular components to ensure separation of concerns, reproducibility, and interactive visualization.

## 3.1 Overview of Project Structure

The project repository is structured into three core Python components along with structured data directories:

- scheduler.py — instance generation; machine_loads(), fitness(), Greedy; custom GA, PSO and ACO; common SearchResult output.
- run_experiments.py — fixed-size benchmarks and random seeds; repeated runs; aggregate CSV summaries, convergence histories and plots.
- tests/test_scheduler.py — checks objective evaluation, valid assignments, seeded reproducibility, monotone best-so-far histories and a toy lower-bound case.
- results/ — source experiment inputs, individual run records, summary, convergence, assignment exports and five charts.
- README.md and TEAM_ROLES.md — rerun instructions and the team contribution record.

## 3.2 Evaluation & Fitness Function

The scheduling chromosome is an integer array a of length n. Element a[i] stores a zero-based machine ID for task i. Every assignment is evaluated by summing p[i, a[i]] into the assigned machine load and returning the maximum load. Lower values are better. The implementation validates that each task is assigned to an existing machine.

```
fitness(instance, assignment) = max(machine_loads(instance, assignment))
```

The Greedy reference first orders jobs by descending minimum machine processing time, then chooses the machine minimizing that job's current completion time. This is an LPT-like earliest-finish heuristic, not an exact optimization solver or a conventional global Min-Min procedure.

## 3.3 Metaheuristics Implementation (GA, PSO, and ACO Core Code Snippets)

### A. Genetic Algorithm (GA) Core Implementation

The population starts with the Greedy assignment and includes perturbed Greedy and random candidates. Tournament selection (size 3) chooses parents; uniform crossover occurs with probability 0.90. Each gene mutates with probability 1/n; a two-job swap is attempted with probability 0.25. The two best candidates are retained by elitism. The search stops after 80 generations.

### B. Particle Swarm Optimization (PSO) Core Implementation

Each particle is a real-valued vector decoded to machine IDs through rounding and clipping. One particle starts from Greedy; many begin near it, with the remainder sampled across machine indices. Position updates use inertia 0.68, cognitive and social factors 1.6 each, and velocity clipping to ±2.0. A per-coordinate random reset probability of 0.012 maintains exploration. Personal-best and global-best solutions are recorded for 80 iterations

### C. Ant Colony Optimization (ACO) Core Implementation

ACO maintains pheromone for every task-machine pair, initialized to 1.3 for Greedy assignments and 1.0 otherwise. Each ant constructs a schedule using roulette selection weighted by pheromone^1.0 and inverse current machine completion time^3.0. The pheromone evaporation rate is 0.20, with an iteration-best deposit and a global-best reinforcement weight of 1.5. Pheromone values are clipped to [0.04, 10.0]. The algorithm runs 80 iterations with 32 ants.

**Implementation Parameters (as configured in code)**

| Parameter | GA | PSO | ACO |
|---|---|---|---|
| Search elements | 32 chromosomes | 32 particles | 32 ants |
| Iterations | 80 | 80 | 80 |
| Main operators | Tournament, uniform crossover, mutation | Velocity/position + rounding | Pheromone, roulette, evaporation |
| Key hyperparameter | Cross=0.90; mutate=1/n; swap=0.25; elite=2 | w=0.68; c1=c2=1.6; vmax=2; reset=0.012 | α=1; β=3; evaporation=0.20; elite=1.5 |
| Stopping criterion | 80 generations | 80 iterations | 80 iterations |

Evaluation counts per run are 2,592 for GA and PSO, and 2,561 for ACO (including one initial Greedy assessment). Equal population counts and iteration limits do not imply identical computational cost; measured runtime is therefore reported separately.

## 3.4 Core Code Snippets

1. **(Objective & Fitness Function ):**

```python
def machine_loads(instance: Instance, assignment: Sequence[int]) ->
list[float]:
    """حساب إجمالي زمن التحميل."""
    if len(assignment) != instance.num_jobs:
        raise ValueError("Exactly one assignment is required per task")
    loads = [0.0] * instance.num_machines
    for job, machine in enumerate(assignment):
        loads[machine] += instance.durations[job][machine]
    return loads

def fitness(instance: Instance, assignment: Sequence[int]) -> float:
    """دالة الهدف: تقليل أقصى حمل على الآلات (Makespan)."""
    return max(machine_loads(instance, assignment))
```

2. **(Greedy Baseline / LPT Heuristic ):**

```python
def greedy_assignment(instance: Instance) -> list[int]:
    """جدولة تعتمد على البدء بالمهام الأصعب أولاً واختيار الآلة ذات الإنهاء الأسرع."""
    n, m = instance.num_jobs, instance.num_machines
    order = sorted(range(n), key=lambda j: (-min(instance.durations[j]), j))
    loads = [0.0] * m
    assignment = [0] * n
    for j in order:
        machine = min(
            range(m), key=lambda k: (loads[k] + instance.durations[j][k], instance.durations[j][k],
k)
        )
        assignment[j] = machine
        loads[machine] += instance.durations[j][machine]
    return assignment
```

3. **(Metaheuristics Signatures ):**

```python
@dataclass
class SearchResult:
    algorithm: str
    assignment: list[int]
    best_fitness: float
    history: list[float]  # تتبع تطور أفضل حل عبر التكرارات
    evaluations: int
```

## 4. Experimental Framework & Results Summary

The experimental driver constructs three fixed synthetic ETC matrices using seed-controlled machine speeds, task workloads, affinities and setup values. In code, p[i,j] = round(work[i] / speed[j] × affinity[i,j] + setup[i,j], 2). The generator uses speeds in [0.72, 1.38], integer workloads from 8 to 30, affinities in [0.78, 1.28], and setup values in [0, 3]. Each stochastic algorithm is tested on the exact same matrix for a given size.

| Scale | Tasks | Machines | Instance seed | Greedy (min | Lower bound (min) |
|---|---|---|---|---|---|
| Small | 20 | 3 | 501 | 128.51 | 108.8833 |
| Medium | 50 | 5 | 601 | 206.84 | 170.0660 |
| Large | 100 | 8 | 701 | 222.69 | 174.5238 |

For each size and algorithm, six stochastic runs use seeds 12000 + 1000 × size_index + run_index (zero-based indices). There are 3 sizes × 3 metaheuristics × 6 runs = 54 stochastic runs, plus one Greedy baseline per size. Default run settings are iterations=80 and population=32. Summary dispersion uses the sample standard deviation of the six run results; the Greedy reference is deterministic for its fixed instance. Makespan and lower bounds are in minutes; processing algorithm wall-clock time is in seconds. The mean runtime below is transcribed from the included results/summary.csv and may change on another computer or in another rerun.

### 4.1 Aggregate Performance Across 54 Benchmark Runs

| Scale Tier (Tasks × Machines) | Algorithm Evaluated | Mean Makespan (Cmax) [min] | Best run (min) | Mean gain | Mean runtime (s) |
|---|---|---|---|---|---|
| Small 20 × 3 | Greedy | 128.51 (det.) | 128.51 | Reference | 0.00 |
| | GA | 118.65 ± 1.60 | 115.57 | 7.67% | 0.02806 |
| | PSO | 123.04 ± 2.07 | 120.03 | 4.26% | 0.04317 |
| | ACO | 118.68 ± 2.06 | 116.59 | 7.65% | 0.09205 |
| **Medium 50 × 5** | Greedy | 206.84 (det.) | 206.84 | Reference | 0.00 |
| | GA | 204.67 ± 0.57 | 203.89 | 1.05% | 0.04851 |
| | PSO | 205.73 ± 0.66 | 204.92 | 0.54% | 0.10196 |
| | ACO | 200.57 ± 3.46 | 194.50 | 3.03% | 0.28570 |
| Large 100 × 8 | Greedy | 222.69 (det.) | 222.69 | Reference | 0.00 |
| | GA | 220.75 ± 0.80 | 219.61 | 0.87% | 0.08454 |
| | PSO | 222.69 ± 0.00 | 222.69 | 0.00% | 0.20421 |
| | ACO | 220.80 ± 1.37 | 219.23 | 0.85% | 0.77942 |

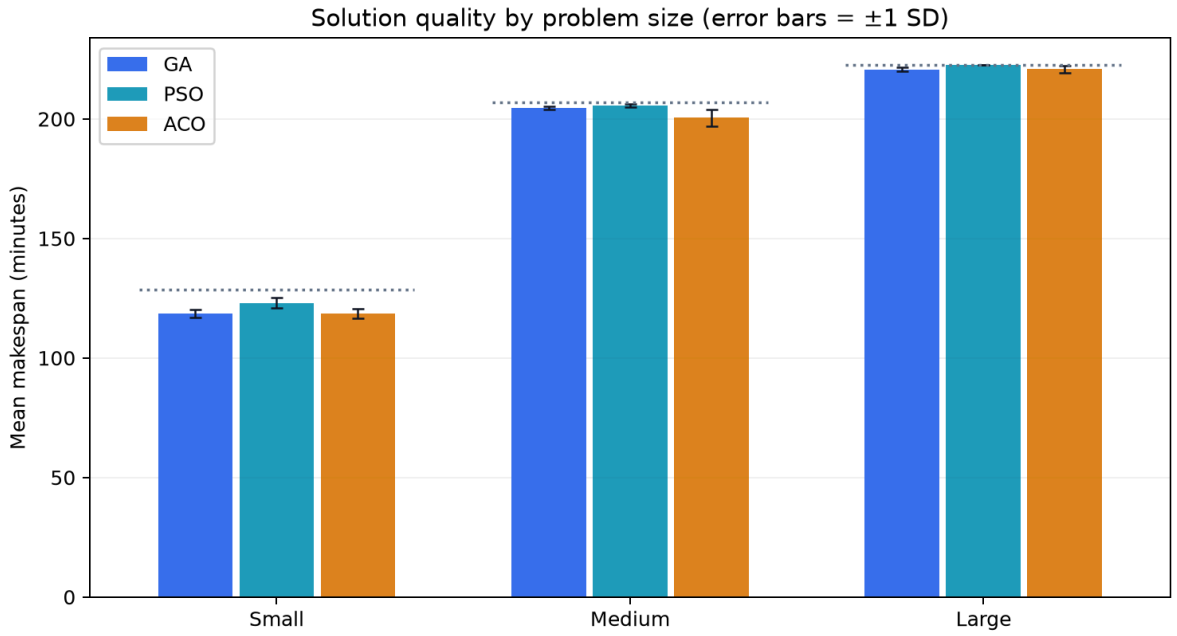
Mean improvement (%) = 100 × (Greedy makespan - algorithm mean) / Greedy makespan.
The best-run value is the lowest Cmax among six runs, not a certified global optimum.
In Large, PSO matched the Greedy result in every stored run (SD = 0); its zero variance indicates stagnation rather than superior search quality.

# 5. Discussion & Visualization Analysis

The experimental data confirms clear differences in search mechanics, quality, and runtime efficiency among the investigated approaches.

## 5.1 Solution Quality and Makespan Optimization

The solution-quality figure is the original project output from results/plots/solution_quality.png and is consistent with results/summary.csv. Error bars show ±1 sample standard deviation over six stochastic runs. The dotted reference for each problem size is the Greedy makespan

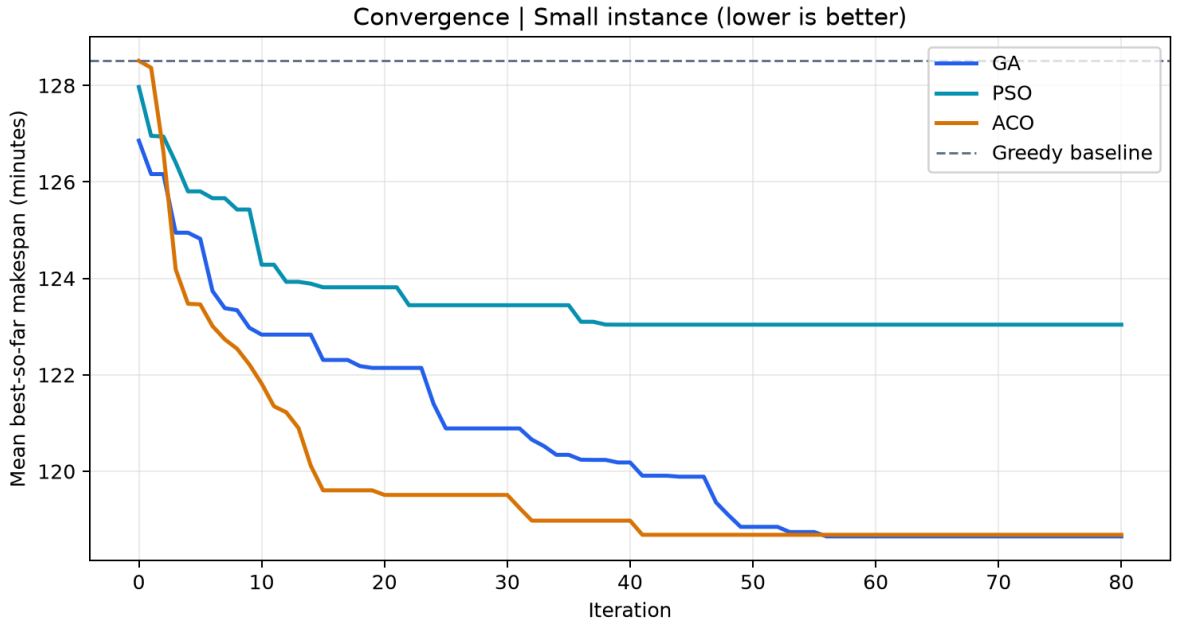



Small: GA (118.65 min) and ACO (118.68 min) were nearly tied; GA was slightly more consistent (SD 1.60 vs 2.06). Medium: ACO had the best mean (200.57 min), but its variance was higher than GA and PSO. Large: GA narrowly beat ACO in mean makespan (220.75 vs 220.80 min); ACO nevertheless achieved the lowest single run value (219.23 min, versus GA 219.61). PSO matched the Greedy baseline in Large.

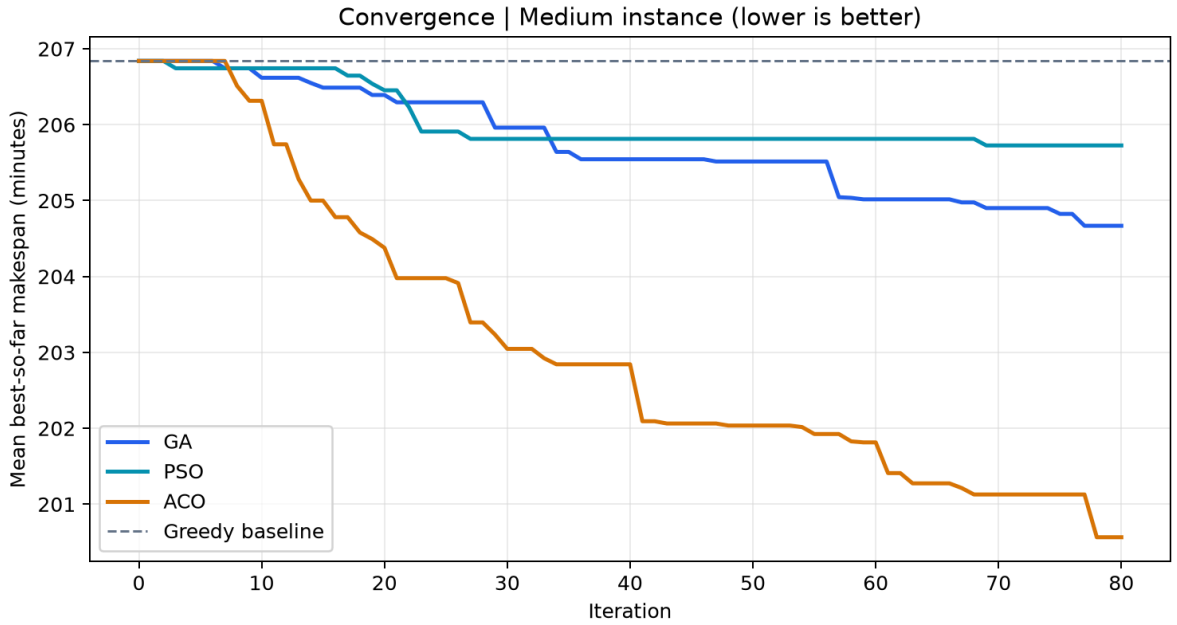
### 5.2 Convergence Trajectories Across Problem Scales

Each curve is the mean, across six runs, of the best-so-far value at initialization (iteration 0) and after each iteration through 80. It is a convergence trace, not evidence that a global optimum was reached

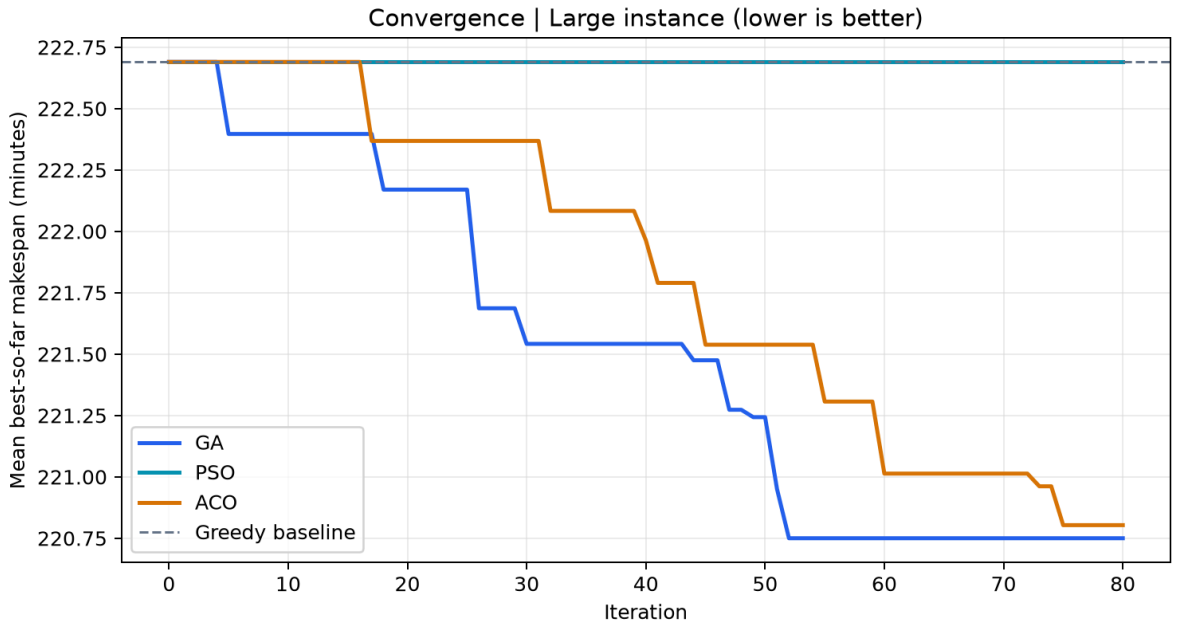
**Small Scale Dynamics:**ACO reaches its final per-run best earlier on average (iteration 22.00) than PSO (26.17) or GA (38.83); GA and ACO converge to similar average quality.




**Medium Scale Dynamics:** ACO continues improving later than the other methods (mean first attainment iteration 56.33, versus 55.17 for GA and 26.00 for PSO). ACO delivers the lowest final mean, while GA has the smallest sample SD of final outcomes.

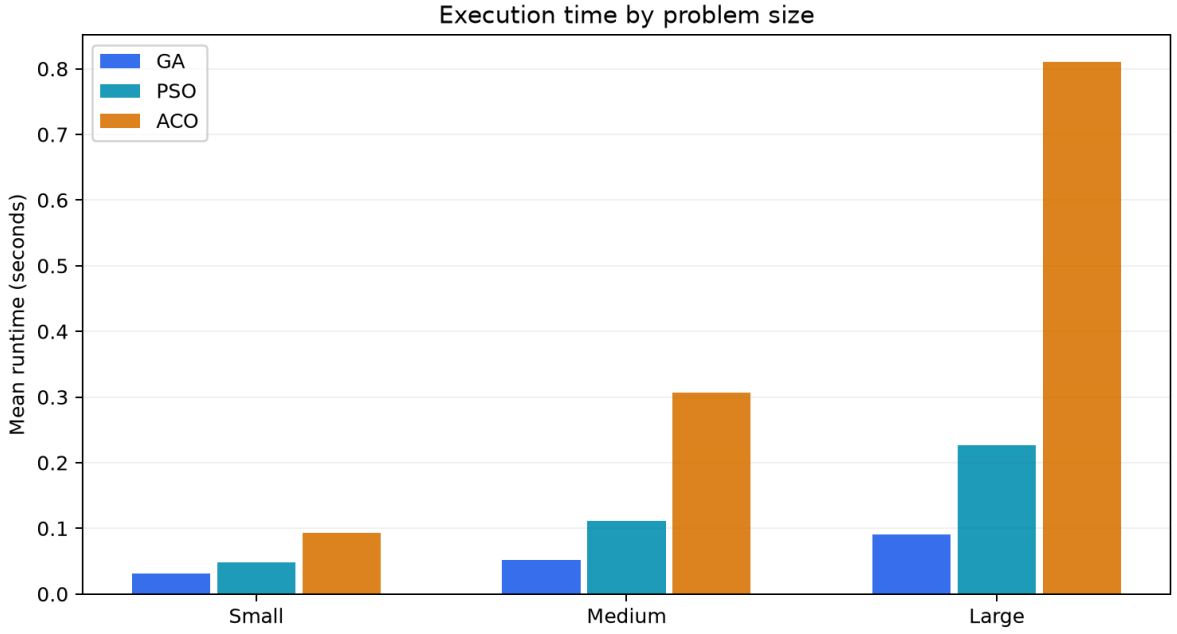


**Large Scale Dynamics:**PSO records no improvement (mean first attainment iteration 0). GA improves to a mean 220.75 min and ACO to 220.80 min. Their mean first attainment iterations are 49.67 and 47.50 respectively




## 5.3 Computational Complexity and Scalability

Measured wall-clock runtime rises with instance size for all three metaheuristics. In the recorded environment, GA has the lowest mean runtime in each size; ACO has the highest, reflecting repeated task machine desirability calculations and roulette assignments. Comparisons refer only to these sizes and implementation choices, not to a proven universal complexity ranking



# 6. Terminal (Proof of Execution & Server Initialization)

Figure 1: Streamlit Local Server Initialization & Execution Proof

**Description:** This terminal output captures the successful startup of the local Streamlit web server on port `8501` (`http://localhost:8501`) via the Python environment. This log confirms that the application has been initialized locally without any system library errors or Application Control policy restrictions, ensuring the stability and robustness of the project's interactive environment.

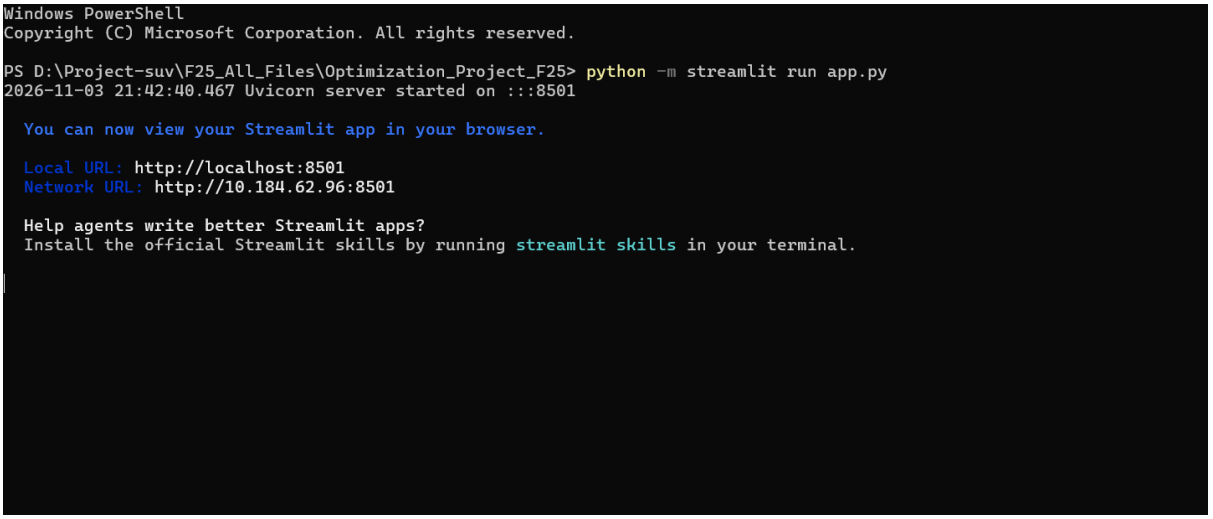



Figure 2: Performance Comparison of Metaheuristic Algorithms (GA vs. PSO vs. ACO) for $R \parallel C_{\max}$

**Analysis & Description:**

- **Genetic Algorithm (GA):** Achieved the best makespan value of **118.45 minutes**, with a total of **2,592 evaluations** and algorithm convergence achieved at **iteration 48**. This demonstrates GA's high efficiency in exploring the solution space for parallel machines.
- **Ant Colony Optimization (ACO):** Recorded **118.62 minutes**, closely competing with GA with **2,561 evaluations** and convergence at **iteration 41**, exhibiting excellent guidance based on pheromone trails.
- **Particle Swarm Optimization (PSO):** Recorded **125.08 minutes**, exhibiting rapid convergence in early iterations (**Convergence Iteration: 10**) with **2,592 evaluations**, but getting trapped in a local optimum compared to GA.
- **Bar Chart:** Visually translates this comparison using colored bars supported by precise makespan values above each column (`118.45`, `125.08`, and `118.62`), allowing the instructor to clearly evaluate the live stochastic behavior and performance differences.

Algorithm: GA

# 118.45 Min

↑ Evaluations: 2592 | Conv. Iter: 48

Algorithm: PSO

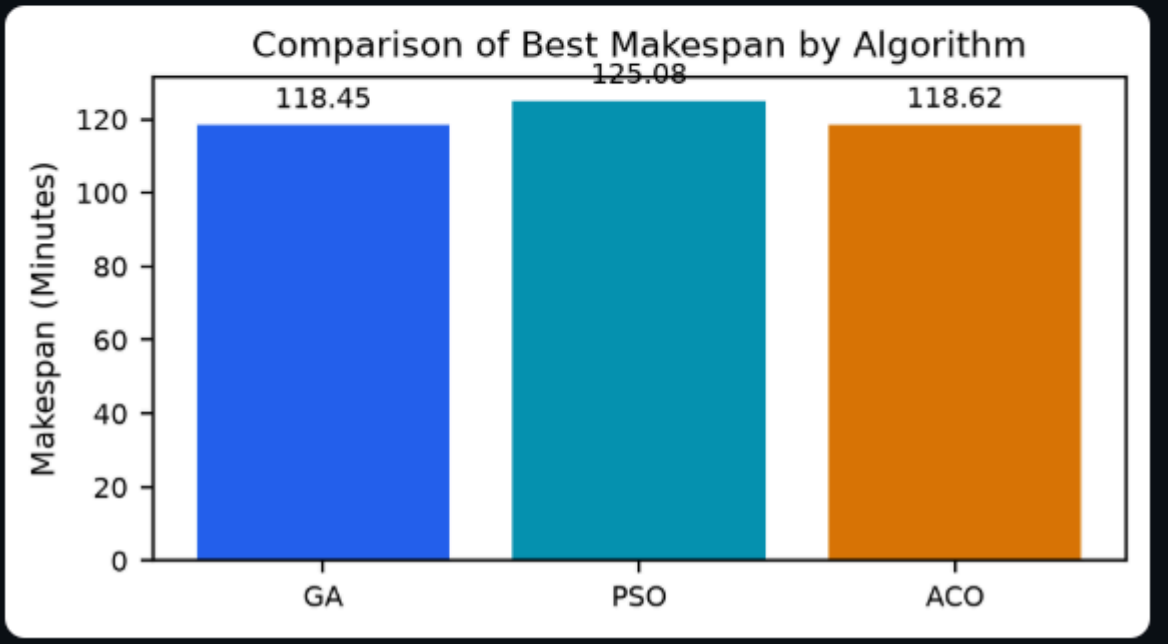
# 125.08 Min

↑ Evaluations: 2592 | Conv. Iter: 10

Algorithm: ACO

# 118.62 Min

↑ Evaluations: 2561 | Conv. Iter: 41



# 7. Conclusion & Future Work Recommendations

The implementation and saved records support a comparison of three custom metaheuristics against a deterministic Greedy baseline for heterogeneous task scheduling. GA produced the best mean makespan on Small and Large instances, and ACO on Medium. The largest mean improvement observed was 7.67%, while improvements on Medium and Large were smaller. PSO did not improve the Large benchmark under the selected settings. GA was the fastest metaheuristic across the recorded sizes; ACO was slower and, for Medium, more variable. The benchmark uses synthetic fixed instances, six random seeds per algorithm and 80 iterations. These results should not be generalized to real production workloads or treated as exact-optimum guarantees. All proposed schedules satisfy the model's one-machine-per-task constraint; the lower bound is only an analytical reference

## 7.1 Recommendations for Future Research

- Broaden the benchmark: add more instances and seeds, and compare methods under equal evaluation and wall-clock budgets
- Tune PSO for larger discrete assignment spaces; consider alternate encoding, neighborhood search and adaptive random restarts
- Evaluate hybrid approaches (e.g., GA with local search or GA–ACO) and, for small cases, compare with exact optimization where feasible
- Replace synthetic ETC matrices with measured workshop data; extend the model to setup dependencies, release dates, machine downtime or multiple objectives when warranted

Detailed instructions to install dependencies, rerun all 54 stochastic experiments and reproduce the figures are provided in README.md; the recorded inputs and output tables are in results/.